

Mycellius — Séance 4 (4h) : Spring Boot #1

API REST minimale + tests Postman

Objectif de la séance (ce qu'on veut réussir)

À la fin de cette séance, tu dois avoir :

- ton module Maven api transformé en application Spring Boot qui démarre sur <http://localhost:8080>
- un endpoint /api/health qui permet de vérifier que l'API répond

À quoi sert vraiment /api/health ?

L'idée de ce endpoint n'est pas fonctionnelle (métier), mais technique.

Il sert à :

- Vérifier que l'application est démarrée.
- Vérifier que le serveur HTTP répond.
- Donner un point d'entrée très simple pour :
 - les ops / admin systèmes,
 - les sondes de supervision (monitoring),
 - parfois les load balancers (test de "liveness" / "readiness").

Typiquement :

- Un script de supervision fait régulièrement un GET sur /api/health.
- Si ça répond 200 OK avec un JSON correct, on considère que l'application est "en vie".
- Si ça renvoie une erreur (timeout, 5xx, etc.), on peut déclencher une alerte.

Donc ce n'est pas un endpoint "métier Mycellius", c'est un endpoint "technique", pour savoir si l'API est en bon état.

Pourquoi ce nom "health" ?

Parce que dans le monde des API / microservices, c'est une convention ultra courante :

/health

/healthcheck

/actuator/health (avec Spring Boot Actuator)

L'avantage :

Tous les développeurs / ops comprennent immédiatement ce que c'est.

- un contrôleur REST qui expose le cœur du service WikiService de la séance 3 :
 - POST /api/pages : créer une page en mémoire
 - GET /api/pages/{id} : récupérer une page par id
 - GET /api/pages/search?title=fragment : rechercher par titre
- une gestion simple des statuts HTTP en fonction des exceptions de la séance 3 :
 - 201 Created quand une page est créée
 - 200 OK sur les lectures qui se passent bien
 - 400 Bad Request pour les IllegalArgumentException (données invalides)
 - 404 Not Found pour PageNotFoundException
- une mini-batterie de requêtes Postman pour tester tout ça manuellement
- la CI GitHub qui continue à lancer mvn -f api/pom.xml test sans rien casser

Plan détaillé de la séance 4 (4h)

1. Rappel séance 3 et objectif séance 4 (≈ 10–15 min)
2. Préparer le module api pour Spring Boot (≈ 45–60 min)
3. Concevoir les endpoints HTTP à partir de WikiService (≈ 20–30 min)
4. Créer le contrôleur REST + endpoint /health + endpoints pages (≈ 60–70 min)
5. Mapper les exceptions vers les statuts HTTP (200, 201, 400, 404) (≈ 45–50 min)
6. Tester l'API avec Postman (≈ 40–50 min)
7. CI + workflow Git dev → test (≈ 20–30 min)

1. Rappel séance 3 et positionnement de la séance 4

Fin de séance 3, tu as dans le module api :

- le modèle métier : Tag, WikiPage
- un dépôt en mémoire : InMemoryWiki (Map<String, WikiPage>)
- un service métier : WikiService avec au moins :
 - createPage(id, title, content, tags...)
 - getPageById(id)
 - searchByTitle(fragment)
- des exceptions métier : PageNotFoundException (et IllegalArgumentException déjà utilisées)
- des tests JUnit sur WikiService (cas nominal + cas d'erreur)
- une CI GitHub qui lance mvn -f api/pom.xml test sur dev / test / prod

Idée importante de séance 4 :

On ne réécrit pas le métier. On “met une couche HTTP” par-dessus le service existant.

En Java :

- niveau métier : WikiService (déjà testé)
- niveau API : un contrôleur Spring qui reçoit des requêtes HTTP, appelle WikiService, renvoie des réponses JSON + statuts HTTP.

2. Préparer le module api pour Spring Boot (≈ 45–60 min)

2.1 Vérifier le pom.xml actuel

Dans api/pom.xml, tu as aujourd’hui quelque chose de très simple de type “projet Maven standard” avec :

- groupId fr.mycellius
- artifactId api
- des dépendances JUnit, peut-être des plugins Maven de base

Spring Boot va s’appuyer sur ce pom.xml, mais en ajoutant un parent Spring et quelques dépendances.

2.2 Ajouter Spring Boot dans pom.xml

Objectif

Transformer le module api en application Spring Boot web minimale.

Où ?

Fichier api/pom.xml

Étape 1 – Déclarer le parent Spring Boot

Dans la balise <project>, juste après <modelVersion>, ajoute un parent Spring Boot (ou remplace

le parent existant si tu en as un).

Exemple :

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.2.0</version>
  <relativePath/> <!-- recherche le parent dans le repo central -->
</parent>
```

Étape 2 – Vérifier les coordonnées du module

Assure-toi d'avoir bien :

```
<groupId>fr.mycllius</groupId>  
<artifactId>api</artifactId>  
<version>0.0.1-SNAPSHOT</version>  
<name>mycllius-api</name>
```

=>Le nom est libre, mais ça aide à s'y retrouver.

Étape 3 – Ajouter les dépendances Spring Boot web et test

Dans la section <dependencies>, ajoute :

```
<dependencies>  
  <!-- Spring Web pour exposer des endpoints HTTP REST -->  
  <dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-web</artifactId>  
  </dependency>  
  
  <!-- Tests Spring Boot (on gardera aussi JUnit qui est déjà là) -->  
  <dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-test</artifactId>  
    <scope>test</scope>  
  </dependency>  
  
  <!-- Tes dépendances déjà présentes (JUnit, etc.) peuvent rester ici -->  
</dependencies>
```

Étape 4 – Ajouter le plugin Maven Spring Boot

Ajoute la section <build> :

Tu la rajoutes au même niveau que <dependencies>, **juste après </dependencies>** et avant </project>

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

À quoi sert ce plugin spring-boot-maven-plugin ?

Concrètement, il permet :

- de lancer l'appli avec : `mvn -f api/pom.xml spring-boot:run`
- de construire un jar "exécutable" correctement packagé pour Spring Boot avec :
`mvn -f api/pom.xml package`

Sans ce plugin, beaucoup de choses Spring Boot continuent à marcher (le parent en configure déjà une partie), mais le déclarer clairement dans <build> rend le TP plus lisible pour apprendre :

"C'est ce plugin-là qui sait démarrer notre API Spring Boot via Maven."

Explications rapides :

- **spring-boot-starter-parent :**

donne des versions cohérentes pour les libs Spring.

- **spring-boot-starter-web :**

apporte un serveur web embarqué (Tomcat par défaut) et Spring MVC.

- **spring-boot-starter-test :**

fournit JUnit, AssertJ, MockMvc... pour tester l'API.

- **spring-boot-maven-plugin :**

permet d'exécuter l'appli avec `mvn spring-boot:run` et de créer un jar exécutable.

une fois que tu as mis à jour le pom, on fait, en ligne de commande, un petit :

```
mvn -f api/pom.xml clean test
```

```
mvn -f api/pom.xml spring-boot:run
```

pour vérifier que tout est OK mais normalement, tu vas avoir des erreurs donc je te laisse déboguer et comprendre pourquoi ça bug...

Teste ton debuggage en relançant les 2 commandes précédentes

Quand tu es à cette étape, appelle-moi pour m'expliquer le bug et comment tu as fait pour résoudre le problème.

2.3 Créer la classe d'entrée Spring Boot

Objectif

Remplacer l'ancien main "console" par un main Spring Boot (ou en créer un nouveau) qui démarre un serveur HTTP.

Où ?

Dans `api/src/main/java/fr/mycellius`, crée un fichier `MycelliusApiApplication.java`

Mais avant de créer ce nouveau fichier, on repose les choses pour mieux comprendre...

Actuellement, tu as dans ton module api :

- une classe `App` avec un main "console" (séance 2),
- et le plan de séance 4 te dit : créer une classe `MycelliusApiApplication` pour être le point d'entrée Spring Boot.

Spring Boot, lui, veut **UNE** seule méthode main dans `src/main/java`. Donc il faut choisir qui sera "le chef d'orchestre" :

- soit `App`,
- soit `MycelliusApiApplication`.

Comme le support de séance 4 parle de `MycelliusApiApplication`, le plus propre pour le cours, c'est d'utiliser :

- `MycelliusApiApplication` = point d'entrée Spring Boot (officiel),
- `App` = éventuellement une petite classe de démo SANS main (ou supprimée).

L'idée :

On ne jette pas la pédagogie de la séance 2, on change juste le nom de la classe qui porte main pour la suite.

Je te propose donc ce plan très concret. Le But final (pour être claire) est :

- une seule méthode main dans tout `src/main/java`,
- elle sera dans `fr.mycellius.MycelliusApiApplication`,
- `App.java` n'aura plus de main, ou sera supprimée => **dans notre cas, on va supprimer `App.java`.**

Tout ce qu'on a expliqué sur "public static void main(String[] args)" reste vrai, c'est juste que la classe s'appelle maintenant `MycelliusApiApplication` au lieu de `App`.

Étape 1 – Créer MycelliusApiApplication (classe Spring Boot)

Dans IntelliJ, dans api/src/main/java/fr/mycellius :

- clic droit sur le package fr.mycellius
- New → Java Class
- Nom : MycelliusApiApplication

Dans le fichier MycelliusApiApplication.java, mets ce contenu :

```
package fr.mycellius;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class MycelliusApiApplication {

    public static void main(String[] args) {
        SpringApplication.run(MycelliusApiApplication.class, args);
    }
}
```

Explication :

@SpringBootApplication : dit à Spring Boot “c’est la classe principale, scanne fr.mycellius.* pour trouver les composants”.

public class MycelliusApiApplication devient donc la classe principale de l’appli.

main : point d’entrée Java comme on l’a vu en séance 2, mais au lieu de System.out.println seulement, on appelle SpringApplication.run pour démarrer tout le contexte Spring + le serveur HTTP.

En d’autres termes, **SpringApplication.run** démarre Spring, charge le contexte, puis lance le serveur web sur un port (par défaut 8080).

Tu peux garder App.java pour des mini-demos console, mais à partir de maintenant, le “vrai” lancement de l’API se fera via MycelliusApiApplication.

2.4 Premier démarrage Spring Boot

Dans un terminal, à la racine du repo (dossier mycellius) :

```
mvn -f api/pom.xml spring-boot:run
```

Résultat attendu dans la console :

- beaucoup de logs Spring
- à la fin, une ligne du type “Started MycelliusApiApplication in ... seconds”
- “Tomcat started on port 8080 (http)”

```

      _____
     / ____ \   | |  | |
    / / ___ \|  | |  | |
   / /  _ \|  | |__| |
  /_/____\_\_|_____|_|

:: Spring Boot ::      (v2.0.9)

2026-01-17T20:47:15.059+01:00 INFO 34084 ---[main] fr.myclillus.MyclillusApiApplication : Starting MycillusApiApplication using Java 21.0.9 with PID 34084 (C:\Users\clakg\mycelius\api\target\classes started by clakg in C:\Users\clakg\mycelius\api)
2026-01-17T20:47:15.065+01:00 INFO 34084 ---[main] fr.myclillus.MyclillusApiApplication : No active profile set, falling back to 1 default profile: "default"
2026-01-17T20:47:15.071+01:00 INFO 34084 ---[main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-01-17T20:47:15.085+01:00 INFO 34084 ---[main] o.apache.catalina.core.StandardService : Starting service [tomcat]
2026-01-17T20:47:15.088+01:00 INFO 34084 ---[main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.16]
2026-01-17T20:47:15.097+01:00 INFO 34084 ---[main] m.o.a.c.c.f.TomcatContextHartl[] : Initializing Spring embedded WebApplicationContext
2026-01-17T20:47:15.097+01:00 INFO 34084 ---[main] w.s.o.s.ServletWebServerApplicationContext : Root webApplicationContext: initialization completed in 748 ms
2026-01-17T20:47:16.104+01:00 INFO 34084 ---[main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path ''
2026-01-17T20:47:16.140+01:00 INFO 34084 ---[main] fr.myclillus.MyclillusApiApplication : Started MycillusApiApplication in 1.406 seconds (process running for 1.697)

```

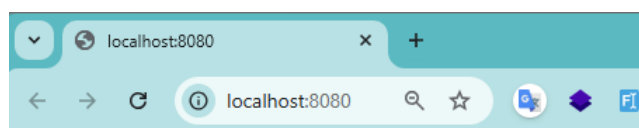
Suite à cette commande, tu auras sûrement une autorisation Java à valider et une synchronisation de maven pour cette nouvelle classe pour faire disparaître ce qui apparaît en erreur.

Vérification rapide dans un autre terminal :

```
curl http://localhost:8080
```

Tu obtiendras probablement une réponse 404 ou une page d'erreur JSON Spring par défaut :
c'est normal, on n'a pas encore d'endpoint personnalisé. Ha ha ha !

```
PS C:\Users\clakg\mycellius> curl http://localhost:8080
curl : Le serveur distant a retourné une erreur : (404) Introuvable.
Au caractère Ligne:1 : 1
+ curl http://localhost:8080
~
+ CategoryInfo          : InvalidOperation : (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-WebRequest], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeWebRequestCommand
```



Whitelabel Error Page

This application has no explicit mapping for /error, so you are seeing this as a fallback.

Sun Jan 11 21:07:28 CET 2026

There was an unexpected error (type=Not Found, status=404).

3. Concevoir les endpoints HTTP à partir de WikiService (≈ 20–30 min)

Objectif

Traduire le contrat Java du service en contrat HTTP.

Rappel WikiService :

- WikiPage createPage(String id, String title, String content, List<Tag> tags)
- WikiPage getPageById(String id)
- List<WikiPage> searchByTitle(String fragment)

Côté HTTP, on veut quelque chose comme :

- POST /api/pages
 - Request body en JSON : id, title, content, tags
 - Réponse : 201 Created, corps = page créée en JSON
- GET /api/pages/{id}
 - Réponse : 200 OK + JSON si trouvé
 - 404 Not Found si PageNotFoundException
- GET /api/pages/search?title=fragment
 - Réponse : 200 OK + liste JSON (possiblement vide)

Et en bonus simple :

- GET /api/health
 - Réponse : 200 OK + petit JSON {"status":"UP"}

Idée importante :

On ne renvoie pas des String “brutes”, mais du JSON. Spring Boot se charge de la **sérialisation** (conversion objet → JSON) tant qu’on renvoie des objets Java ou des collections.

4. Créer le contrôleur REST + endpoint /health + endpoints pages (≈ 60–70 min)

4.1 Créer le package web

Pour séparer les couches proprement :

- fr.mycellius.domain : modèle métier + exceptions
- fr.mycellius.repository : InMemoryWiki
- fr.mycellius.service : WikiService
- fr.mycellius.web : contrôleurs REST

Où ?

Dans src/main/java/fr/mycellius, crée un package fr.mycellius.web

4.2 Créer un contrôleur de santé HealthController

Fichier : src/main/java/fr/mycellius/web/HealthController.java

```
package fr.mycellius.web;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.time.Instant;
import java.util.Map;

@RestController
public class HealthController {

    @GetMapping("/api/health")
    public Map<String, Object> health() {
        return Map.of(
            "status", "UP",
            "timestamp", Instant.now().toString(),
            "application", "mycellius-api"
        );
    }
}
```

Explication :

- **@RestController** : indique à Spring que cette classe expose des endpoints REST (renvoie du JSON).
- **@GetMapping("/api/health")** : mappe les requêtes GET /api/health sur cette méthode.
- **health()** renvoie un Map<String,Object> → Spring le convertit automatiquement en JSON.

Test rapide (après démarrage spring-boot:run) :

curl <http://localhost:8080/api/health>

=> Réponse attendue : JSON avec status = UP et un timestamp.

```
PS C:\Users\clakg\mycellius> curl http://localhost:8080/api/health

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
RECOMMENDED ACTION:
  Use the -UseBasicParsing switch to avoid script code execution.

Do you want to continue?

[0] Oui [T] Oui pour tout [N] Non [U] Non pour tout [S] Suspendre [?] Aide (la valeur par défaut est « N ») : 0

StatusCode      : 200
StatusDescription :
Content         : {"application":"mycellius-api","status":"UP","timestamp":"2026-01-11T20:50:00.403326200Z"}
RawContent      : HTTP/1.1 200
                  Transfer-Encoding: chunked
                  Keep-Alive: timeout=60
                  Connection: keep-alive
                  Content-Type: application/json
                  Date: Sun, 11 Jan 2026 20:50:00 GMT

                  {"application":"mycellius-api","status...
Forms           : {}
Headers         : {[Transfer-Encoding, chunked], [Keep-Alive, timeout=60], [Connection, keep-alive], [Content-Type,
                  application/json]...}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 90
```

4.3 Créer un contrôleur pour les pages : WikiPageController

Objectif

Exposer les méthodes de WikiService.

Fichier : src/main/java/fr/mycellius/web/WikiPageController.java

```

package fr.mycellius.web;

import fr.mycellius.domain.WikiPage;
import fr.mycellius.domain.Tag;
import fr.mycellius.service.WikiService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/pages")
public class WikiPageController {

    private final WikiService wikiService;

    public WikiPageController(WikiService wikiService) {
        this.wikiService = wikiService;
    }

    @PostMapping
    public ResponseEntity<WikiPage> createPage(@RequestBody CreatePageRequest request) {
        WikiPage page = wikiService.createPage(
            request.id(),
            request.title(),
            request.content(),
            request.tags()
        );
        return ResponseEntity.status(HttpStatus.CREATED).body(page);
    }

    @GetMapping("/{id}")
    public WikiPage getPageById(@PathVariable String id) {
        return wikiService.getPageById(id);
    }
}

```

```

    @GetMapping("/search")
    public List<WikiPage> searchByTitle(@RequestParam("title") String fragment) {
        return wikiService.searchByTitle(fragment);
    }
}

```

Ici, on utilise un petit DTO pour la requête de création (CreatePageRequest) pour ne pas exposer directement le constructeur de WikiPage.

Avant d'aller plus loin, un DTO c'est quoi ?

DTO = Data Transfer Object (Objet de transfert de données)

Un DTO est une petite classe qui sert uniquement à transporter des données d'un point A à un point B, sans logique métier "intelligente" dedans.

Dans une API REST (comme ton API Mycellius), le DTO est souvent :

- le format des données qui arrive depuis le client (POST, PUT...)
- ou le format des données qu'on renvoie au client (GET, etc.)

Exemple dans Mycellius :

- le client (Postman, front web, mobile...) envoie un JSON pour créer une page du wiki
- ce JSON est converti par Spring en un objet Java = un DTO
- ce DTO est utilisé par le contrôleur pour appeler le service métier

Pourquoi on ne passe pas directement une WikiPage (ton modèle métier) depuis l'API HTTP ?

Raisons principales :

- Séparer le "contrat HTTP" du "modèle métier"
 - Contrat HTTP = ce qu'attend/retourne l'API (champs, formats, etc.)
 - Modèle métier = comment toi tu représentes les concepts dans ton code (WikiPage, Tag, etc.)

On veut pouvoir faire évoluer l'un sans tout casser dans l'autre.

- Protéger le métier
 - Peut-être que ta WikiPage a des champs internes qu'on ne veut pas exposer au client (dates techniques, flags internes, etc.)
 - Le DTO ne contient que les champs que tu acceptes d'exposer.

- Gérer les variantes
 - DTO de création : par exemple CreatePageRequest (ce que l'utilisateur envoie pour créer une page)
 - DTO de réponse : par exemple WikiPageResponse (ce que tu renvoies après création ou lecture)
 - Tu peux avoir moins de champs en entrée, plus de champs en sortie, des formats différents, etc.
- Stabiliser l'API
 - Si un jour tu refactorises ton modèle WikiPage (ajout, renommage de champs, changement de structure), tu peux garder les mêmes DTO pour ne pas casser les clients externes.

Donc, dans Mycellius :

- côté HTTP = DTO (CreatePageRequest, WikiPageResponse...)
- côté métier = classes de domaine (WikiPage, Tag, InMemoryWiki, WikiService...)

Le contrôleur Spring sert d'adaptateur entre les deux :

- il reçoit un DTO → appelle ton service métier en lui donnant ce qu'il lui faut
- il récupère un WikiPage → le transforme éventuellement en DTO de réponse → le renvoie au client

Crée ce DTO juste à côté, dans le même package :

```
package fr.mycellius.web;
```

```
import fr.mycellius.domain.Tag;
```

```
import java.util.List;
```

```
public record CreatePageRequest(
```

```
    String id,
```

```
    String title,
```

```
    String content,
```

```
    List<Tag> tags
```

```
) {}
```

Explications :

- **@RequestMapping("/api/pages")** au niveau de la classe : préfixe commun pour tous les endpoints → /api/pages.
- **@PostMapping** sans chemin supplémentaire → POST /api/pages.
- **@RequestBody CreatePageRequest request** : Spring lit le JSON du corps, le transforme en objet CreatePageRequest.
- **ResponseEntity.status(HttpStatus.CREATED).body(page)** : permet de renvoyer explicitement le code HTTP 201.
- **@GetMapping("/{id}") + @PathVariable String id** : récupère la variable de chemin {id}
- **@GetMapping("/search") + @RequestParam("title")** : lit le paramètre de query ?title=

Vous aurez des bugs avec Tags... N'hésitez pas à me dire quand vous serez rendu là :)

5. Mapper les exceptions vers les statuts HTTP (≈ 45–50 min)

Objectif

Ne plus renvoyer les 500 Internal Server Error par défaut quand une exception métier se produit, mais renvoyer des codes 400/404 cohérents, avec un message lisible.

5.1 Rappel des exceptions existantes

Dans la séance 3, tu as :

- **IllegalArgumentException** : pour des données invalides (id vide, titre vide...)
- **PageNotFoundException** : pour une page introuvable

=> Le service WikiService les lance déjà.

On va créer un “traducteur” global exceptions → réponses HTTP.

5.2 Créer un type de réponse d’erreur simple

Package : fr.mycellius.web

Fichier : ApiErrorResponse.java

```
package fr.mycellius.web;
```

```
import java.time.Instant;
```

```
public record ApiErrorResponse(  
    String error,  
    String message,  
    int status,  
    String path,  
    String timestamp  
) {  
    public static ApiErrorResponse of(String error, String message, int status, String path) {  
        return new ApiErrorResponse(  
            error,  
            message,  
            status,  
            path,  
            Instant.now().toString()  
        );  
    }  
}
```

```

        path,
        Instant.now().toString()
    );
}
}

```

5.3 Créer un handler global des exceptions avec @RestControllerAdvice

Fichier : src/main/java/fr/mycellius/web/ApiExceptionHandler.java

```

package fr.mycellius.web;

import fr.mycellius.domain.exception.PageNotFoundException;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import jakarta.servlet.http.HttpServletRequest;

@RestControllerAdvice
public class ApiExceptionHandler {

    @ExceptionHandler(PageNotFoundException.class)
    public ResponseEntity<ApiErrorResponse> handlePageNotFound(
        PageNotFoundException ex,
        HttpServletRequest request
    ) {
        ApiErrorResponse body = ApiErrorResponse.of(
            "PAGE_NOT_FOUND",
            ex.getMessage(),
            HttpStatus.NOT_FOUND.value(),
            request.getRequestURI()
        );
    }
}

```

```

        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(body);
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<ApiErrorResponse> handleIllegalArgumentException(
        IllegalArgumentException ex,
        HttpServletRequest request
    ) {
        ApiErrorResponse body = ApiErrorResponse.of(
            "INVALID_INPUT",
            ex.getMessage(),
            HttpStatus.BAD_REQUEST.value(),
            request.getRequestURI()
        );
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(body);
    }
}

```

Explications :

- `@RestControllerAdvice` : classe qui “écoute” les exceptions lancées par les contrôleurs REST.
- `@ExceptionHandler(PageNotFoundException.class)` : cette méthode sera appelée automatiquement si une `PageNotFoundException` remonte.
- `HttpServletRequest request` : permet de récupérer l’URL qui a posé problème (`getRequestURI()`).
- On construit une `ApiErrorResponse` cohérente et on renvoie le bon statut HTTP.

Avec ça :

- une page introuvable renverra un 404 + JSON d’erreur lisible.
- une donnée invalide renverra un 400 + JSON d’erreur.

6. Tester l'API avec Postman (≈ 40–50 min)

Objectif

On va se donner un rituel complet pour tester l'API à la main avant même d'écrire des tests automatisés de type MockMvc.

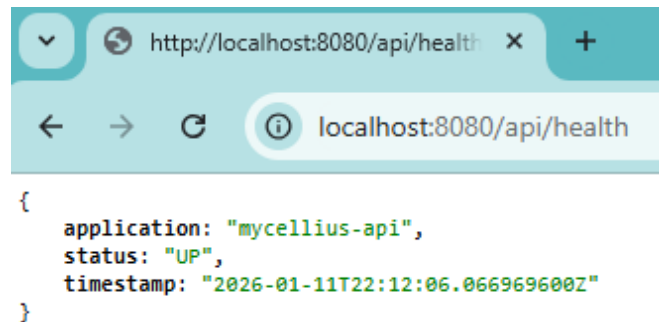
Pré-requis :

- Lancer l'appli Spring Boot depuis la racine du projet :

```
mvn -f api/pom.xml spring-boot:run
```

- Vérifier /api/health avec curl (ou navigateur) :

<http://localhost:8080/api/health>



6.1 Créer une collection Postman "Mycellius – local"

À faire dans Postman :

- Créer une nouvelle Collection :
Nom : Mycellius – Local API
- Ajouter une variable d'environnement simple (facultatif mais propre) :
– baseUrl = <http://localhost:8080>

6.2 Requête 1 : GET /api/health

Dans la collection :

- New Request → nom : Health check
- Méthode : GET
- URL : {{baseUrl}}/api/health
- Send

Résultat attendu :

- Statut 200 OK
- Corps JSON avec "status": "UP"

6.3 Requête 2 : POST /api/pages (cas nominal)

Toujours dans la collection :

- New Request → nom : Create page OK
- Méthode : POST
- URL : {{baseUrl}}/api/pages
- Onglet Body → Raw → JSON

Exemple de corps JSON :

```
{
  "id": "PAGE-001",
  "title": "Bienvenue dans Mycellius",
  "content": "Première page de test.",
  "tags": [
    { "name": "onboarding" },
    { "name": "dev" }
  ]
}
```

puis Send.

Résultat attendu :

- Statut 201 Created
- Corps JSON représentant la page créée (structure de WikiPage)
- Optionnel : vérifier que les tags ont été normalisés si ton modèle le fait.

6.4 Requête 3 : GET /api/pages/{id}

Créer une requête :

- Nom : Get page by id OK
- Méthode : GET
- URL : {{baseUrl}}/api/pages/PAGE-001
- Send.

Résultat attendu :

- Statut 200 OK
- Corps JSON de la page PAGE-001.

6.5 Requête 4 : GET /api/pages/{id} — page introuvable

Créer une requête :

- Nom : Get page not found
- Méthode : GET
- URL : {{baseUrl}}/api/pages/INCONNUE
- Send.

Résultat attendu :

- Statut 404 Not Found
- Corps JSON de type ApiResponse, par exemple :
 - error : "PAGE_NOT_FOUND"
 - message : "Page introuvable pour l'id: INCONNUE"
 - status : 404
 - path : "/api/pages/INCONNUE"

6.6 Requête 5 : GET /api/pages/search?title=...

Créer une requête :

- Nom : Search pages by title
- Méthode : GET
- URL : {{baseUrl}}/api/pages/search?title=bienvenue
- Send.

Résultat attendu :

- Statut 200 OK
- Corps JSON : tableau de pages, possiblement vide, possiblement avec PAGE-001.

6.7 Requête 6 : POST /api/pages avec données invalides

Objectif : montrer le 400 Bad Request.

Exemple de JSON invalide (titre vide) :

```
{  
  "id": "PAGE-002",  
  "title": "",  
  "content": "Page avec titre vide",  
  "tags": []  
}
```

Si WikiService ou WikiPage vérifient bien le titre, on doit obtenir :

- Statut 400 Bad Request
- Corps ApiErrorResponse avec error = "INVALID_INPUT"

7. CI + workflow Git dev → test (≈ 20–30 min)

La CI GitHub (ci.yml) ne change pas beaucoup :

- elle lance toujours `mvn -f api/pom.xml test`
- Spring Boot a ajouté des libs, mais les tests unitaires existants + éventuels tests Spring Boot tourneront dans la même commande.

Rappel du rituel en ligne de commande (sans utiliser l'interface Git d'IntelliJ) :

Sur ton poste (branche dev) :

- Se positionner sur dev et récupérer les dernières modifications :

```
git checkout dev
git pull origin dev
```

Coder dans api/ (IntelliJ) :

- modifications pom.xml
- MycelliusApiApplication, HealthController, WikiPageController
- ApiErrorResponse, ApiExceptionHandler
- éventuels ajustements dans WikiService si nécessaire.

Vérifier que le code compile et que les tests passent :

```
mvn -f api/pom.xml test
```

Préparer le commit :

```
git status
git add api/
git commit -m "feat(api): expose WikiService via Spring Boot REST"
```

Pousser sur dev :

```
git push origin dev
```

Sur GitHub :

- Onglet Actions : vérifier que la CI est verte sur la branche dev.
- Créer une Pull Request :
 - base : test
 - compare : dev

- Vérifier que les checks CI sont en vert.
- Merger la PR vers test si tout est OK.

Optionnel :

- créer un tag v0.1.0 après merge sur test pour marquer “Mycellius après séance 4 (API REST minimale en mémoire + endpoints testés avec Postman)”.

Conclusion de la séance 4

À ce stade, on a :

- un noyau métier testé (séance 3) exposé via une vraie API REST Spring Boot
- des endpoints clairs pour créer, lire et rechercher des pages en mémoire
- une première gestion des statuts HTTP basée sur leurs exceptions métier
- une collection Postman qui sert de “laboratoire” pour tester et montrer l’API
- un workflow Git + CI inchangé, mais désormais autour d’une application Spring complète

La séance 5 pourra s’appuyer directement sur cette base pour :

- brancher l’API sur une vraie base de données (MySQL + JPA)
- gérer les migrations de schéma (Flyway/Liquibase)
- commencer à persister réellement les pages du wiki.